

Visão Geral

Este projeto utiliza um microcontrolador ESP32 para monitorar a umidade através de um potenciômetro. Os dados de umidade são enviados via MQTT para um broker MQTT público, permitindo o monitoramento remoto da umidade. O projeto também inclui um display LCD I2C para exibir informações locais sobre a conexão WiFi e MQTT.

Estrutura do Código

O código está organizado em várias seções principais:

1. **Configurações Iniciais**
2. **Conexões Hardware**
3. **WiFi**
4. **MQTT**
5. **Hardware (Sensores e Atuadores)**
6. **Display LCD I2C**
7. **Funções de Conexão WiFi**
8. **Funções de Conexão MQTT**
9. **Leitura do Sensor de Umidade**
10. **Inicialização**
11. **Loop Principal**

1. Configurações Iniciais

```
# =====  
  
# WIFI  
  
# =====  
  
SSID = "Wokwi-GUEST"  
  
SENHA = ""  
  
  
  
# =====  
  
# MQTT  
  
# =====
```

```
BROKER = "test.mosquitto.org" # Teste com outro broker

CLIENT_ID = f"sigh_mackenzie_luiz_2026_{int(time.time())}" # Adicione um suporte
ao horário para garantir a unicidade

TOPICO_UMIDADE = b"mackenzie/luizferreira/projeto2026/umidade"

TOPICO_COMANDO = b"mackenzie/luizferreira/projeto2026/comando"

TOPICO_STATUS = b"mackenzie/luizferreira/projeto2026/status"
```

```
# =====
```

```
# HARDWARE
```

```
# =====
```

```
sensor = ADC(Pin(34))
```

```
sensor.atten(ADC.ATTN_11DB)
```

```
valvula = Pin(5, Pin.OUT)
```

```
valvula.value(1)
```

```
# =====
```

```
# LCD I2C
```

```
# =====
```

```
i2c = I2C(0, scl=Pin(22), sda=Pin(21), freq=400000)
```

```
devices = i2c.scan()
```

```
if len(devices) == 0:
```

```
    raise Exception("LCD I2C não encontrado")
```

```
addr = devices[0]
```

```
BACKLIGHT = 0x08
```

```
ENABLE = 0x04
```

- **WiFi:** Define as configurações de conexão WiFi.
- **MQTT:** Configurações do broker MQTT, client ID e tópicos utilizados.
- **Hardware:** Inicializa o sensor ADC no pino 34 e a valvula no pino 5.

- **LCD I2C:** Configura a comunicação I2C com o display LCD.

2. Funções de Display LCD

```
def lcd_write(data):
```

```
    i2c.writeto(addr, bytearray([data | BACKLIGHT]))
```

```
def lcd_toggle(data):
```

```
    lcd_write(data | ENABLE)
```

```
    time.sleep_us(1)
```

```
    lcd_write(data & ~ENABLE)
```

```
    time.sleep_us(50)
```

```
def lcd_send_nibble(nibble):
```

```
    lcd_write(nibble)
```

```
    lcd_toggle(nibble)
```

```
def lcd_send_byte(byte, mode):
```

```
    high = mode | (byte & 0xF0)
```

```
    low = mode | ((byte << 4) & 0xF0)
```

```
    lcd_send_nibble(high)
```

```
    lcd_send_nibble(low)
```

```
def lcd_cmd(cmd):
```

```
    lcd_send_byte(cmd, 0)
```

```
def lcd_data(data):
```

```
    lcd_send_byte(data, 1)
```

```
def lcd_init():
```

```
time.sleep(0.02)
lcd_send_nibble(0x30)
time.sleep(0.005)
lcd_send_nibble(0x30)
time.sleep(0.001)
lcd_send_nibble(0x30)
lcd_send_nibble(0x20)
lcd_cmd(0x28)
lcd_cmd(0x0C)
lcd_cmd(0x06)
```

```
def lcd_clear():
    lcd_cmd(0x01)
```

```
def lcd_set_cursor(row, col):
    if row == 0:
        addr = 0x80 + col
    elif row == 1:
        addr = 0xC0 + col
    lcd_cmd(addr)
```

- **lcd_write:** Escreve dados no display.
- **lcd_toggle:** Ativa o sinal de habilitação do LCD.
- **lcd_send_nibble:** Envia um nibble (4 bits) para o LCD.
- **lcd_send_byte:** Envia um byte para o LCD, dividindo-o em dois nibbles.
- **lcd_cmd:** Envia comandos para o LCD.
- **lcd_data:** Envia dados para serem exibidos no LCD.
- **lcd_init:** Inicializa o LCD.
- **lcd_clear:** Limpa a tela do LCD.

- **lcd_set_cursor:** Define a posição do cursor no LCD.

3. Funções de Conexão WiFi

```
def conecta_wifi():
    print("Conectando ao WiFi...")
    lcd_set_cursor(0, 0)
    lcd_print("Conectando WiFi")
    sta_if = network.WLAN(network.STA_IF)
    if not sta_if.isconnected():
        sta_if.active(True)
        sta_if.connect(SSID, SENHA)
        while not sta_if.isconnected():
            pass
        print("Conectado com sucesso!")
    lcd_set_cursor(0, 0)
    lcd_print(f"IP: {sta_if.ifconfig()[0]}")
```

- **conecta_wifi:** Conecta o ESP32 à rede WiFi especificada e exibe o IP no LCD.

4. Funções de Conexão MQTT

```
def callback(topic, msg):
    print(f"Recebido {msg} do tópico {topic}")
    if topic == TOPICO_COMANDO:
        if msg == b'ON':
            valvula.value(1)
        elif msg == b'OFF':
            valvula.value(0)

def conecta_mqtt():
    global client
```

```
print("Conectando ao broker MQTT...")

lcd_set_cursor(1, 0)

lcd_print("Conectando MQTT")

client = MQTTClient(CLIENT_ID, BROKER)

client.set_callback(callback)

client.connect()

client.subscribe(TOPICO_COMANDO)

print("Conectado ao broker!")

lcd_set_cursor(1, 0)

lcd_print("MQTT Conectado")
```

- **callback:** Função chamada quando uma mensagem é recebida em um tópico assinado.
- **conecta_mqtt:** Conecta o ESP32 ao broker MQTT e assina o tópico de comando.

5. Leitura do Sensor de Umidade

```
def ler_umidade():

valor = sensor.read()

umidade = (valor / 4096) * 100 # Mapeia o valor do ADC para uma escala de 0 a 100

return umidade
```

- **ler_umidade:** Lê o valor do potenciômetro e mapeia-o para uma escala de 0 a 100.

6. Inicialização

```
def inicializa():

lcd_init()

conecta_wifi()

conecta_mqtt()
```

- **inicializa:** Inicializa o display LCD, conecta ao WiFi e conecta ao broker MQTT.

7. Loop Principal

```
def loop():  
    while True:  
        umidade = ler_umidade()  
        print(f"Umidade: {umidade:.2f}%")  
        client.publish(TOPICO_UMIDADE, f"{umidade:.2f}")  
        lcd_set_cursor(0, 1)  
        lcd_print(f"Umidade: {umidade:.2f}%")  
        client.check_msg() # Verifica se há mensagens MQTT  
        time.sleep(5)
```

```
if __name__ == "__main__":
```

```
    inicializa()
```

```
    loop()
```

- **loop:** Executa continuamente, lendo a umidade do sensor, publicando o valor no broker MQTT e exibindo-o no LCD. Também verifica se há mensagens MQTT recebidas.

Explicação Detalhada das Funções

1. lcd_write(data)

- **Descrição:** Escreve dados no display LCD.
- **Parâmetros:**
 - data: Dados a serem escritos no LCD.

2. lcd_toggle(data)

- **Descrição:** Ativa o sinal de habilitação do LCD para processar os dados enviados.
- **Parâmetros:**
 - data: Dados a serem enviados ao LCD.

3. lcd_send_nibble(nibble)

- **Descrição:** Envia um nibble (4 bits) para o LCD.
- **Parâmetros:**

- nibble: Nibble a ser enviado ao LCD.

4. `lcd_send_byte(byte, mode)`

- **Descrição:** Envia um byte para o LCD, dividindo-o em dois nibbles.
- **Parâmetros:**
 - byte: Byte a ser enviado ao LCD.
 - mode: Modo de operação (comando ou dado).

5. `lcd_cmd(cmd)`

- **Descrição:** Envia comandos para o LCD.
- **Parâmetros:**
 - cmd: Comando a ser enviado ao LCD.

6. `lcd_data(data)`

- **Descrição:** Envia dados para serem exibidos no LCD.
- **Parâmetros:**
 - data: Dados a serem exibidos no LCD.

7. `lcd_init()`

- **Descrição:** Inicializa o LCD com os parâmetros padrão.

8. `lcd_clear()`

- **Descrição:** Limpa a tela do LCD.

9. `lcd_set_cursor(row, col)`

- **Descrição:** Define a posição do cursor no LCD.
- **Parâmetros:**
 - row: Linha onde o cursor será posicionado (0 ou 1).
 - col: Coluna onde o cursor será posicionado.

10. `conecta_wifi()`

- **Descrição:** Conecta o ESP32 à rede WiFi especificada e exibe o IP no LCD.

11. `callback(topic, msg)`

- **Descrição:** Função chamada quando uma mensagem é recebida em um tópico assinado.

- **Parâmetros:**

- topic: Tópico da mensagem recebida.
- msg: Mensagem recebida.

12. conecta_mqtt()

- **Descrição:** Conecta o ESP32 ao broker MQTT e assina o tópico de comando.

13. ler_umidade()

- **Descrição:** Lê o valor do potenciômetro e mapeia-o para uma escala de 0 a 100.
- **Retorno:**
 - umidade: Valor da umidade mapeado entre 0 e 100.

14. inicializa()

- **Descrição:** Inicializa o display LCD, conecta ao WiFi e conecta ao broker MQTT.

15. loop()

- **Descrição:** Executa continuamente, lendo a umidade do sensor, publicando o valor no broker MQTT e exibindo-o no LCD. Também verifica se há mensagens MQTT recebidas.

Exemplo de Uso

1. Monitorar a Umidade:

- Gire o potenciômetro para variar os valores de umidade.
- Os valores atualizados serão exibidos no display LCD I2C e enviados via MQTT.

2. Controlar a Valvula:

- Publique um comando ON no tópico mackenzie/luizferreira/projeto2026/comando.
- A valvula será ativada (GPIO5 alto).
- Publique um comando OFF para desativar a valvula.

Exemplo de Código Completo

```
import network
```

```
from machine import Pin, ADC
```

```
from umqtt.simple import MQTTClient

import time


# =====

# WIFI

# =====

SSID = "Wokwi-GUEST"

SENHA = ""


# =====

# MQTT

# =====

BROKER = "test.mosquitto.org" # Teste com outro broker

CLIENT_ID = f"sigh_mackenzie_luiz_2026_{int(time.time())}" # Adicione um suporte
ao horário para garantir a unicidade

TOPICO_UMIDADE = b"mackenzie/luizferreira/projeto2026/umidade"

TOPICO_COMANDO = b"mackenzie/luizferreira/projeto2026/comando"

TOPICO_STATUS = b"mackenzie/luizferreira/projeto2026/status"


# =====

# HARDWARE

# =====

sensor = ADC(Pin(34))

sensor.atten(ADC.ATTN_11DB)

valvula = Pin(2, Pin.OUT)


# =====

# I2C LCD
```

```

# =====

from machine import I2C

import lcd_i2c

i2c = I2C(scl=Pin(5), sda=Pin(4))

lcd = lcd_i2c.LCD16x2(i2c)

def conecta_wifi():
    print("Conectando ao WiFi...")
    lcd.lcd_clear()
    lcd.lcd_string("Conectando WiFi", 1)
    sta_if = network.WLAN(network.STA_IF)
    if not sta_if.isconnected():
        sta_if.active(True)
        sta_if.connect(SSID, SENHA)
        while not sta_if.isconnected():
            pass
        print("Conectado com sucesso!")
        lcd.lcd_clear()
        lcd.lcd_string(f"IP: {sta_if.ifconfig()[0]}", 1)

def callback(topic, msg):
    print(f"Recebido {msg} do tópico {topic}")
    if topic == TOPICO_COMANDO:
        if msg == b'ON':
            valvula.value(1)
        elif msg == b'OFF':
            valvula.value(0)

```

```
def conecta_mqtt():  
    global client  
    print("Conectando ao broker MQTT...")  
    lcd lcd_string("Conectando MQTT", 2)  
    client = MQTTClient(CLIENT_ID, BROKER)  
    client.set_callback(callback)  
    client.connect()  
    client.subscribe(TOPICO_COMANDO)  
    print("Conectado ao broker!")  
    lcd lcd_string("MQTT Conectado", 2)  
  
def ler_umidade():  
    valor = sensor.read()  
    umidade = (valor / 4096) * 100 # Mapeia o valor do ADC para uma escala de 0 a 100  
    return umidade  
  
def inicializa():  
    lcd lcd_init()  
    conecta_wifi()  
    conecta_mqtt()  
  
def loop():  
    while True:  
        umidade = ler_umidade()  
        print(f"Umidade: {umidade:.2f}%")  
        client.publish(TOPICO_UMIDADE, f"{umidade:.2f}")  
        lcd lcd_string(f"Umidade: {umidade:.2f}%", 1)
```

```
client.check_msg() # Verifica se há mensagens MQTT  
time.sleep(5)
```

```
if __name__ == "__main__":  
    inicializa()  
    loop()
```

Conclusão

Este guia detalhou a implementação de um sistema que lê a umidade através de um potenciômetro, exibe o valor no display LCD I2C e envia os dados via MQTT para um broker. O sistema também pode receber comandos via MQTT para controlar uma valvula conectada ao ESP32. As funções estão bem documentadas e explicadas, facilitando a compreensão e a manutenção do código.

Referências

- [MicroPython Documentation](#)
- [ESP32 Datasheet](#)
- [I2C LCD Library](#)